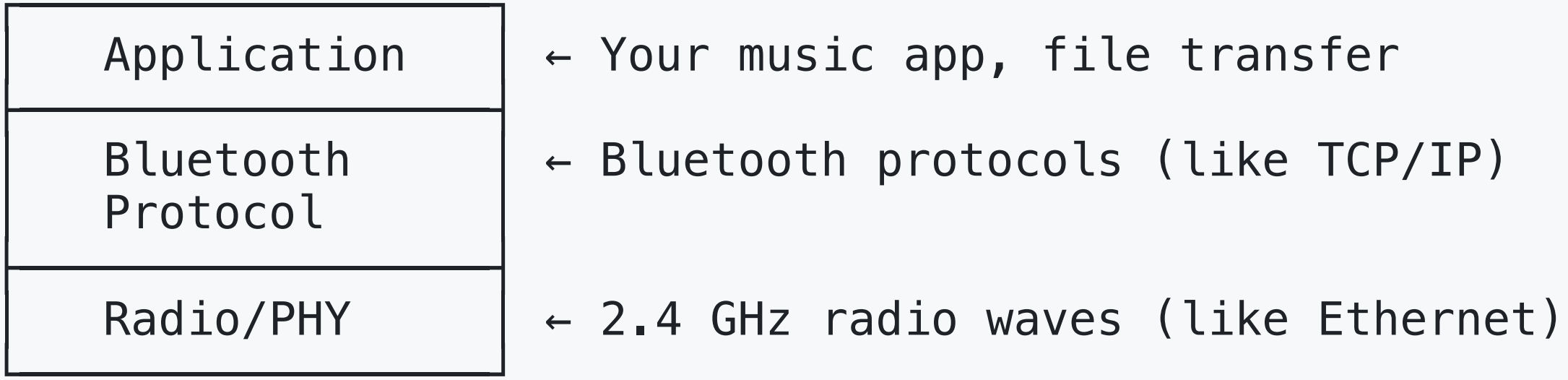
Bluetooth Stack: The Full Architecture

How Bluetooth Works: The Networking Analogy

Think of it like a Network Protocol Stack:



Just like Internet networking, but for short distances!

Bluetooth Protocol Stack vs TCP/IP Stack

Layer	Bluetooth	Rough TCP/IP Analogy
Application	Audio, HID, GATT	Application (HTTP, FTP, etc.)
L2CAP	Logical channel mgmt, segmentation	Transport-ish (Similar to TCP/UDP)
HCI	Host <-> Controller interface	(No direct match, but similar to driver/bus interface)
Link Layer / Baseband	Connection, timing, encryption	Data Link (Ethernet)
PHY	2.4 GHz Radio	Physical Layer

Each layer has specific responsibilities, just like OSI model!

1. Application layer in the Bluetooth Stack already defines the use case (e.g., audio streaming, sensor data) and how data is structured.

- It is not exactly the same as the Application layer in TCP/IP, which is more general-purpose and can run a wide variety of protocols (HTTP, FTP, etc.) on top of it.

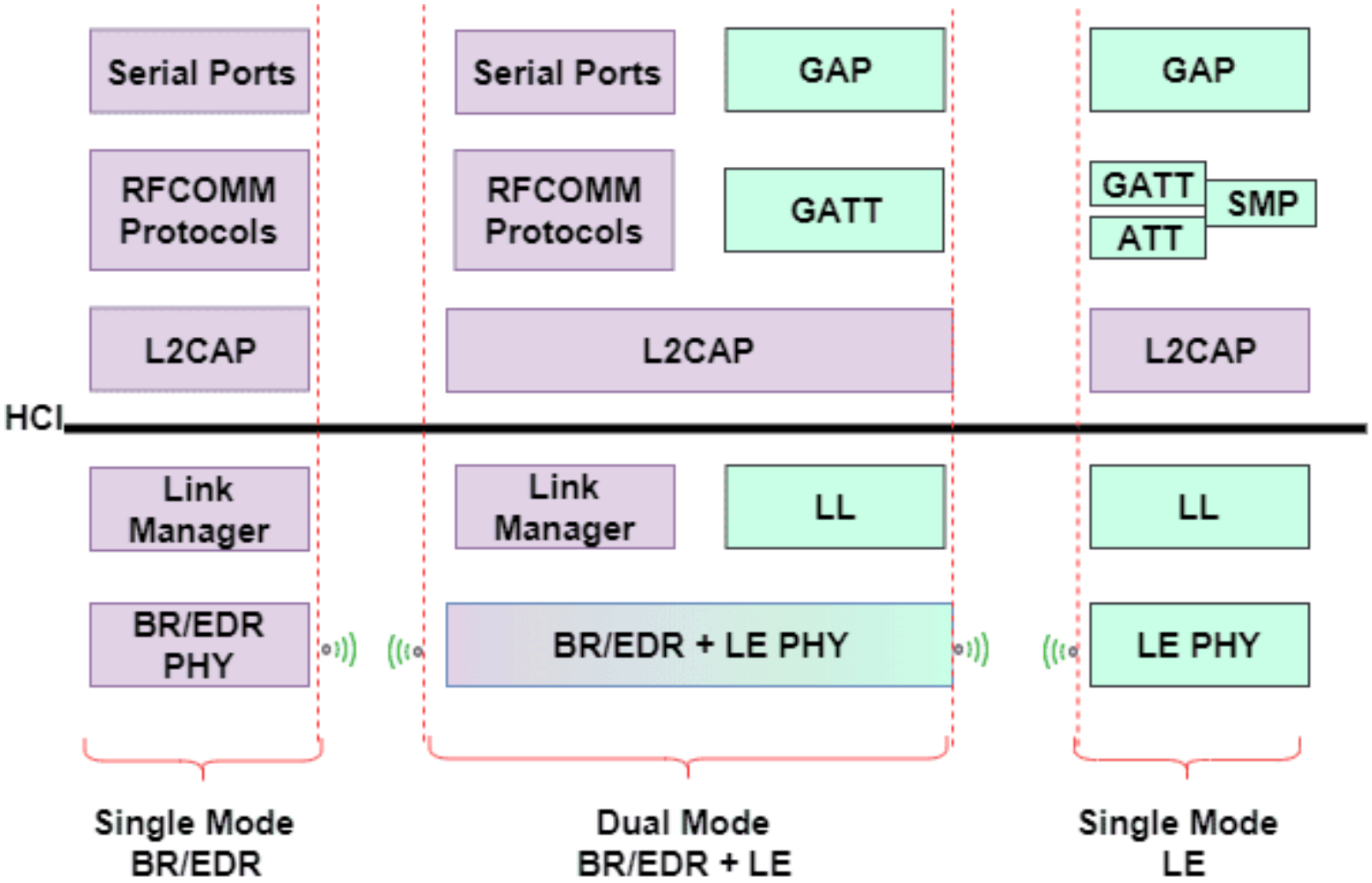
2. L2CAP is more of a transport layer (TCP/UDP) that handles segmentation and multiplexing, but it does not provide the same end-to-end reliability guarantees as TCP.

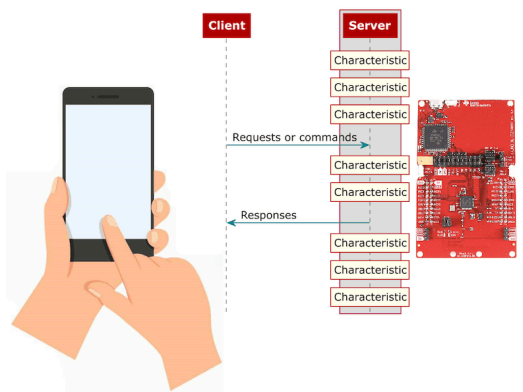
3. HCI is a unique layer that serves as the interface between the Host (software) and Controller (hardware/radio).

- It does not have a direct analogy in TCP/IP, but it is crucial for Bluetooth operation.

4. Bluetooth's Link Layer and PHY are responsible for the actual radio communication, including frequency hopping, timing, and encryption.

- This is somewhat analogous to the Data Link and Physical layers in the OSI model, but with Bluetooth-specific functions.


Serial Ports
RFCOMM Protocols
L2CAP
HCI
Link Manager
BR/EDR PHY
Serial Ports
RFCOMM Protocols
GAP
GATT
L2CAP
Link Manager
LL
BR/EDR + LE PHY
GAP
GATT
SMP
ATT
L2CAP
LL
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE


Client
Server
Characteristic
Characteristic
Characteristic
Requests or commands
Characteristic
Characteristic
Responses
Characteristic
Characteristic
Characteristic

Bluetooth Stack Context

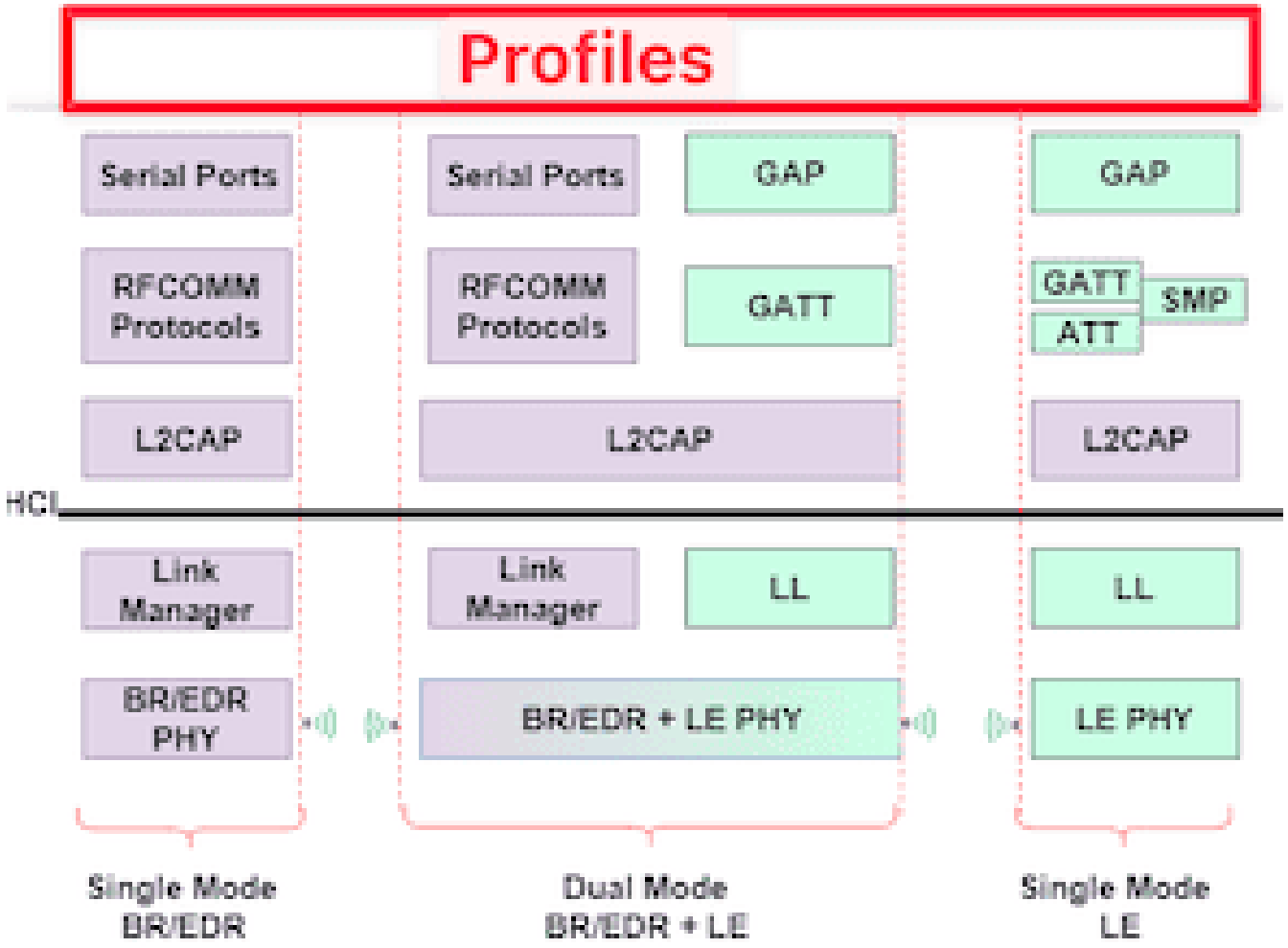
A **stack** is the full communication architecture
that defines *how data moves from application to radio*.

Think of it as:

> The entire layered system that makes Bluetooth work.

Bluetooth Stack for BLE/Classic

Functional Role	BLE Layer	Classic Bluetooth Equivalent	Purpose
Application behavior	Profiles (via GAP)	Profiles (A2DP, SPP, HID)	Defines use cases
Data organization	GATT / ATT	RFCOMM / L2CAP	Data formatting & channels
Packet transport	L2CAP	L2CAP	Segmentation & multiplexing
Host <-> Controller boundary	HCI	HCI	Software <-> radio interface
Link control / timing / hopping	Link Layer	Baseband	RF link operation
Physical transmission	PHY	Radio	2.4 GHz signal transmission


Profiles
Serial Ports
RFCOMM Protocols
L2CAP
Serial Ports
GAP
RFCOMM Protocols
GATT
L2CAP
GAP
GATT
SMP
ATT
L2CAP
HCI
Link Manager
BR/EDR PHY
Link Manager
LL
BR/EDR + LE PHY
LL
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE

Application Layer: Profiles

A **profile** defines how to use Bluetooth
for a specific application.

It sits on top of the stack (Application layer).

Examples:

- A2DP → Audio streaming
- SPP → Serial communication
- HID → Keyboard/Mouse
- GATT → BLE sensor data (GATT is a framework, but often considered part of the application layer)

Simple Analogy

Stack = Operating System
Profile = Application

Stack = Communication engine
Profile = Application rules

- Without stack → nothing works
- Without profile → no defined behavior so we need to make all the rules ourselves

The Problem Without Profiles

Imagine connecting devices with no predefined behavior:

- You must install drivers
- Configure data formats
- Map controls manually
- Handle compatibility issues

Result:

> Connection works... but setup is painful.

Example 1 — Wireless Headphones

Without Profile Support

- Manually select audio codec
- Configure stereo channels
- Map play/pause buttons

> Not user-friendly.

With Profiles (A2DP + AVRCP)

Profiles automatically provide:

- Audio streaming (A2DP)
- Play/Pause/Next controls (AVRCP)

Result:

> Connect → Music plays → Buttons work instantly
> Notice that users do NOT manually select profiles.

How It Works

When two Bluetooth devices connect:

1. They exchange capability information
2. Each device lists supported profiles
3. They find the common profiles
4. The appropriate profile is activated

This is called **profile negotiation**.

Profiles define HOW devices communicate for specific tasks:

Profile	Purpose	Real-world Example
A2DP	High-quality audio	Spotify → Bluetooth headphones
HID	Human Interface Device	Bluetooth keyboard/mouse
OBEX	File transfer	Sending photos between phones
HSP	Headset communication	Phone calls with Bluetooth earpiece
SPP	Serial communication	Arduino <-> Computer data exchange

Example 1 - Serial Communication (SPP)

Without SPP

- You must define your own packet format
- Handle addressing and retransmissions
- Implement your own protocol logic

With SPP

- SPP provides a virtual serial port
- You can send bytes as if over UART
- No need to reinvent the wheel

Example 2 — Car Hands-Free System

User Expectation

- Auto connect when car starts
- Route calls to speakers
- Use steering-wheel buttons

Profile Support: Hands Free Protocol (HFP)

Provides:

- Call audio routing
- Microphone activation
- Answer/Reject controls

Result:

> No setup. Just drive and talk.

Example 3 — Keyboard Typing

User Expectation

Keyboard connects to laptop.

But:

- How are keystrokes encoded?
- How are special keys mapped?

Profile Support: HID Profile

Defines:

- Keycode format
- Input reports
- Device roles

Result:

> Any Bluetooth keyboard works instantly.

BLE Profiles (GATT Framework)

BLE profiles are built on top of GATT, which defines how data is structured and accessed.

BLE profiles define:

- Required **services**
- Required **characteristics**
- Data format
- Device behavior

They are built **on top of GATT**.

Examples of BLE Profiles

BLE Profile	Use Case
Heart Rate Profile	Fitness monitors
Health Thermometer	Temperature sensors
Battery Service	Battery level reporting
HID over GATT	Keyboard / mouse
Proximity Profile	Anti-loss / alerts
Glucose Profile	Medical devices

How BLE Profiles Work

Instead of streaming bytes (like Classic),

BLE profiles define:

- Services (groups of data)
- Characteristics (individual values)
- Read / Write / Notify behavior

Example:

Heart Rate Profile includes:

- Heart Rate Measurement characteristic
- Standard unit (beats per minute)
- Notification structure

Key Difference from Classic

Classic → Profiles built on RFCOMM streams
BLE → Profiles built on GATT attribute model

Classic = Data pipe
BLE = Structured database

BLE profiles standardize

- how structured data is organized and exchanged.
- They ensure interoperability between sensors, apps, and devices.

Then, who makes these profiles?

Bluetooth SIG (Special Interest Group)

They define interoperable standards so devices from different vendors work together.

Examples:

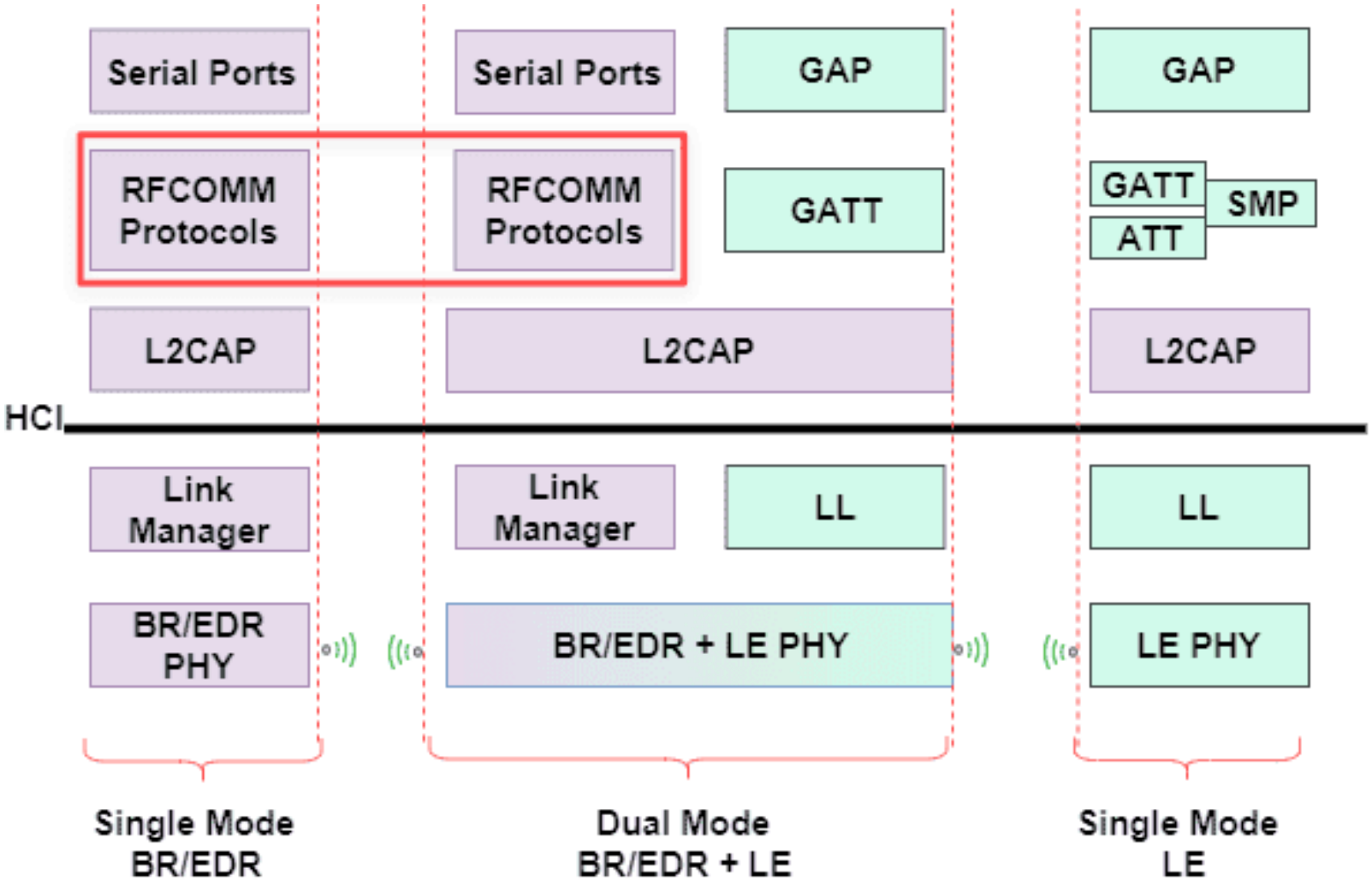
- Heart Rate Profile
- Battery Service
- HID over GATT
- Glucose Profile

If you follow these:

> Any compliant app/device can understand your data.

Data Organization Layer

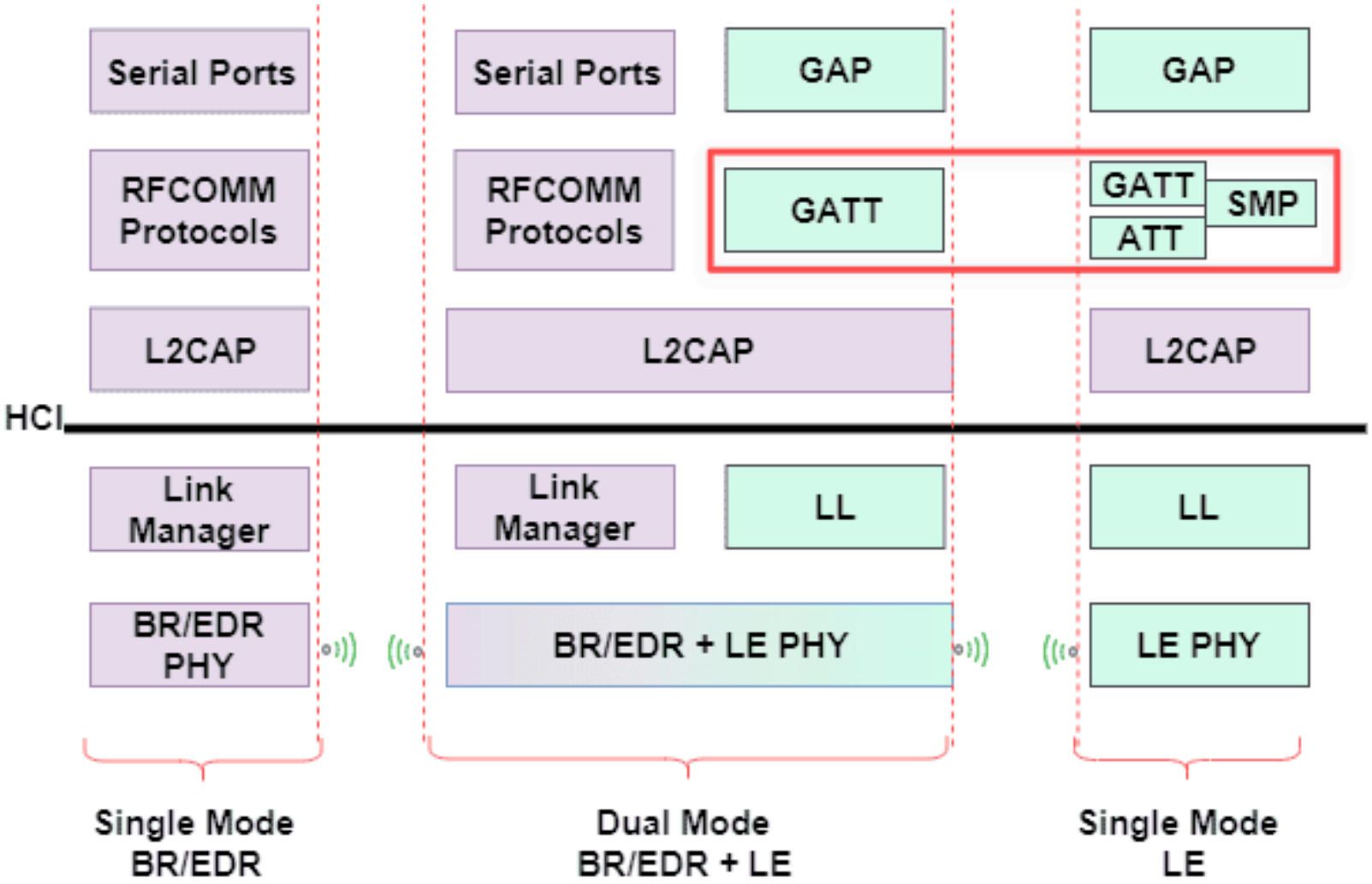
- RFCOMM for Classic Bluetooth
- BLE Frameworks: GATT & GAP


Serial Ports
Serial Ports
GAP
GAP
RFCOMM Protocols
RFCOMM Protocols
GATT
GATT
SMP
ATT
L2CAP
L2CAP
L2CAP
HCI
Link Manager
Link Manager
LL
LL
BR/EDR PHY
BR/EDR + LE PHY
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE

RFCOMM for Classic Bluetooth

RFCOM is a protocol that provides serial communication over Bluetooth.

- The "Serial Port Profile" (SPP) is built on top of RFCOMM
- It emulates a serial cable connection


Serial Ports
RFCOMM Protocols
L2CAP
HCI
Link Manager
BR/EDR PHY
Serial Ports
RFCOMM Protocols
GAP
GATT
L2CAP
Link Manager
LL
BR/EDR + LE PHY
GAP
GATT
SMP
ATT
L2CAP
LL
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE

GATT (Generic ATTribute Profile) for BLE

GATT defines how data is structured and accessed in BLE.

It defines:

- Data structure
- Services
- Characteristics
- Read/Write/Notify operations

Problem GATT Solves

Once connected:

- How is data organized?
- How is it accessed?

GATT standardizes data exchange.

GATT Example — Fitness Tracker App on BLE

User Expectation

Phone must read:

- Heart rate
- Steps
- Calories

Data format varies by vendor.

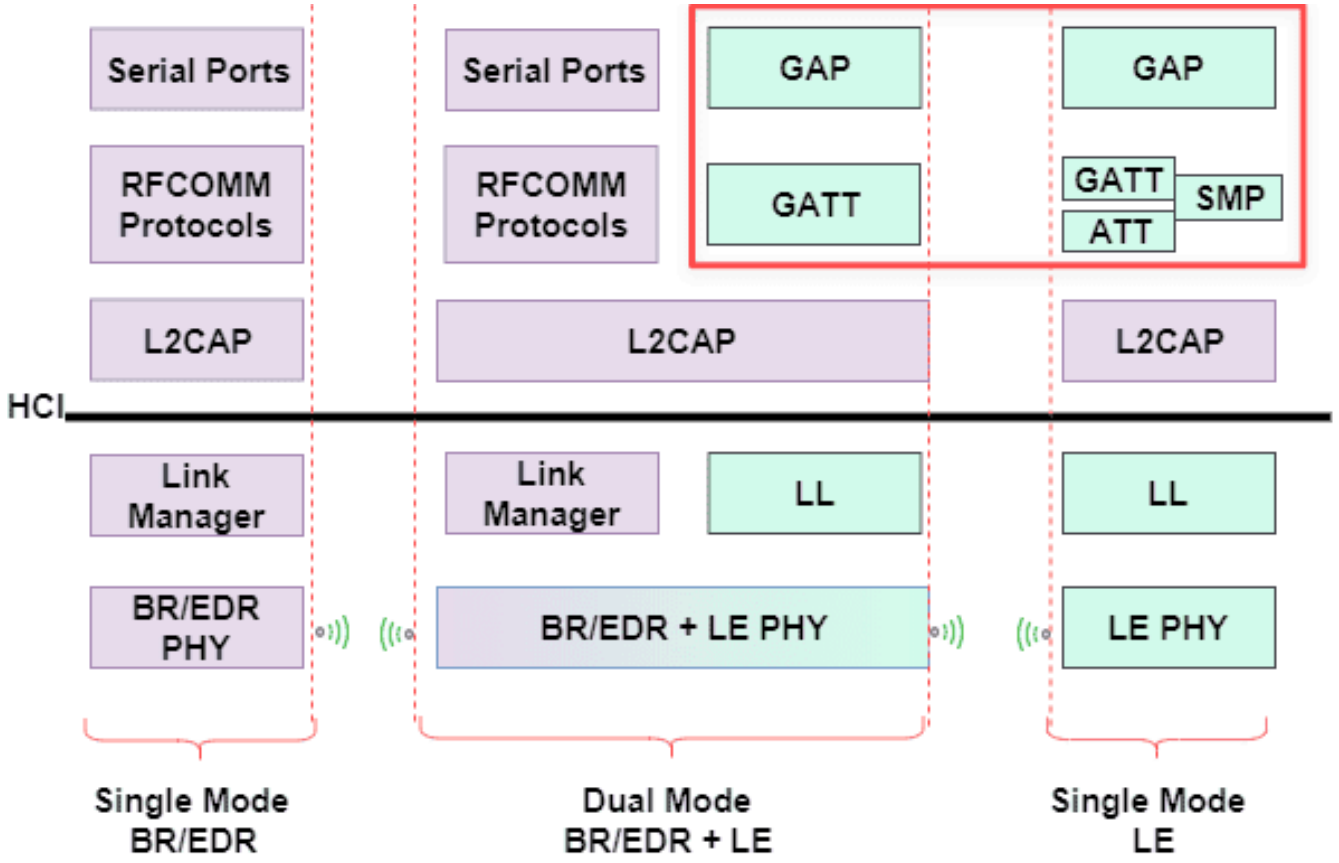
Profile Support: GATT / Health Profiles

Defines:

- Standard sensor data formats
- Measurement units
- Notification structure

Result:

> Any health app can read the data.



GAP applies **horizontally across all Host layers**, not vertically as one layer.

GAP (Generic Access Profile)

GAP is **not a single protocol layer** in the stack.

Instead, it is a **cross-layer policy framework** that sits above HCI and governs how devices interact at the Host level.

> Think of GAP as the **rules of the building**:
> who can enter, how they identify themselves, and what security is required —
> before any actual service (GATT) begins.

What GAP Controls

- **Device Discovery** — advertising, scanning, visibility
- **Device Roles** — Central, Peripheral, Broadcaster, Observer
- **Connection Establishment** — how and when to connect
- **Pairing & Bonding** — security negotiation
- **Privacy** — address randomization to prevent tracking

GAP Device Roles

Role	Description	Example
Central	Scans and initiates connections	Smartphone
Peripheral	Advertises and accepts connections	Heart rate sensor
Broadcaster	Sends data without connecting	BLE beacon
Observer	Listens to advertisements, no connect	Gateway / scanner

GAP vs GATT — The Key Difference

Aspect	GAP	GATT
Purpose	How devices find & connect	How data is organized & exchanged
When	Before and during connection	After connection is established
Analogy	Building security & entry rules	Services available inside

Simple Analogy

GATT = Services inside a building (coffee shop, gym, office)

GAP = Building entrance rules:

- Who can enter?
- Do you need an ID?
- Are you allowed to stay?

Without GAP, no device would know
how to find or trust another device —
even if GATT services are perfectly defined.

Before Connection

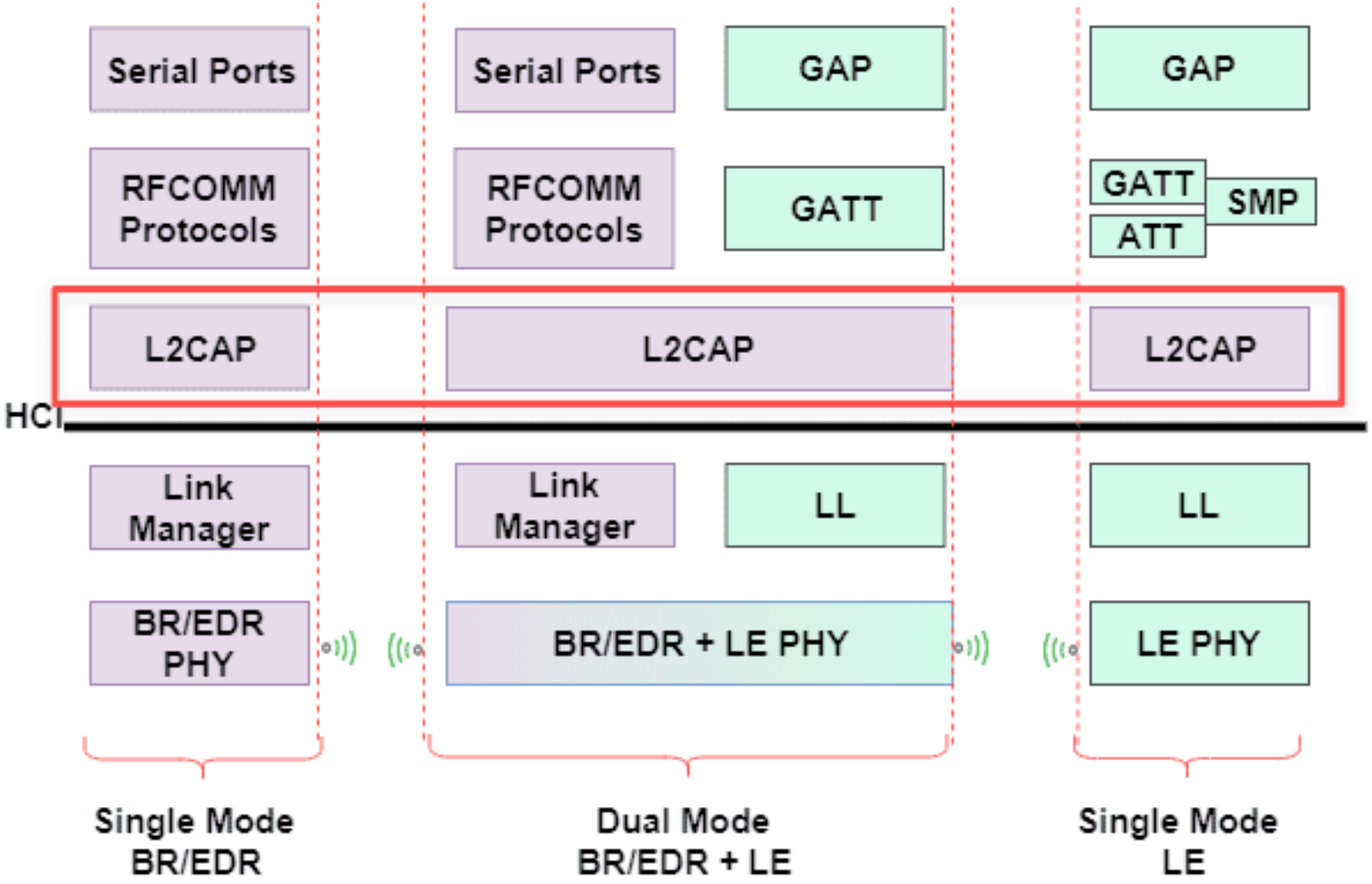
GAP → Advertising / Scanning / Discovery

Connection Setup

GAP → Role negotiation (Central/Peripheral)
GAP → Pairing & Bonding (security keys)

After Connection — Data Exchange

GATT → Read / Write / Notify characteristics
(GAP is checking connection policies in the background)


Serial Ports
RFCOMM Protocols
L2CAP
HCI
Link Manager
BR/EDR PHY
Serial Ports
RFCOMM Protocols
L2CAP
Link Manager
GAP
GATT
LL
BR/EDR + LE PHY
GAP
GATT
SMP
ATT
L2CAP
LL
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE

L2CAP: The Bluetooth Transport Layer

L2CAP is the layer that moves data
between applications and the Bluetooth link.

- Above L2CAP → Meaningful data
- Below L2CAP → 0s and 1s

> Think of it as the **data delivery manager**.

Above L2CAP

Data has **structure and meaning**.

Examples:

- Audio frames/Serial characters
- Heart rate values/Keyboard keycodes

Protocols here define:

- Data format
- Semantics
- Application logic

Below L2CAP

Data becomes **transport packets**.

Handled as:

- Packet headers
- Timing slots
- Frequency hops
- Error correction bits

Focus is on delivery, not meaning.

Simple Analogy

Bluetooth link = Highway
L2CAP = Traffic controller

It:

- Splits cargo
- Assigns lanes
- Reassembles deliveries

So everything arrives correctly.

What L2CAP Does

1. Segmentation & Reassembly

Big data → Split into small packets
Small packets → Reassembled at receiver

Like shipping a sofa in boxes
and rebuilding it at delivery.

2. Logical Channels

Multiple apps share one connection.

Example:

- Music streaming
- Volume control
- Battery status

Each gets its own “virtual lane”.

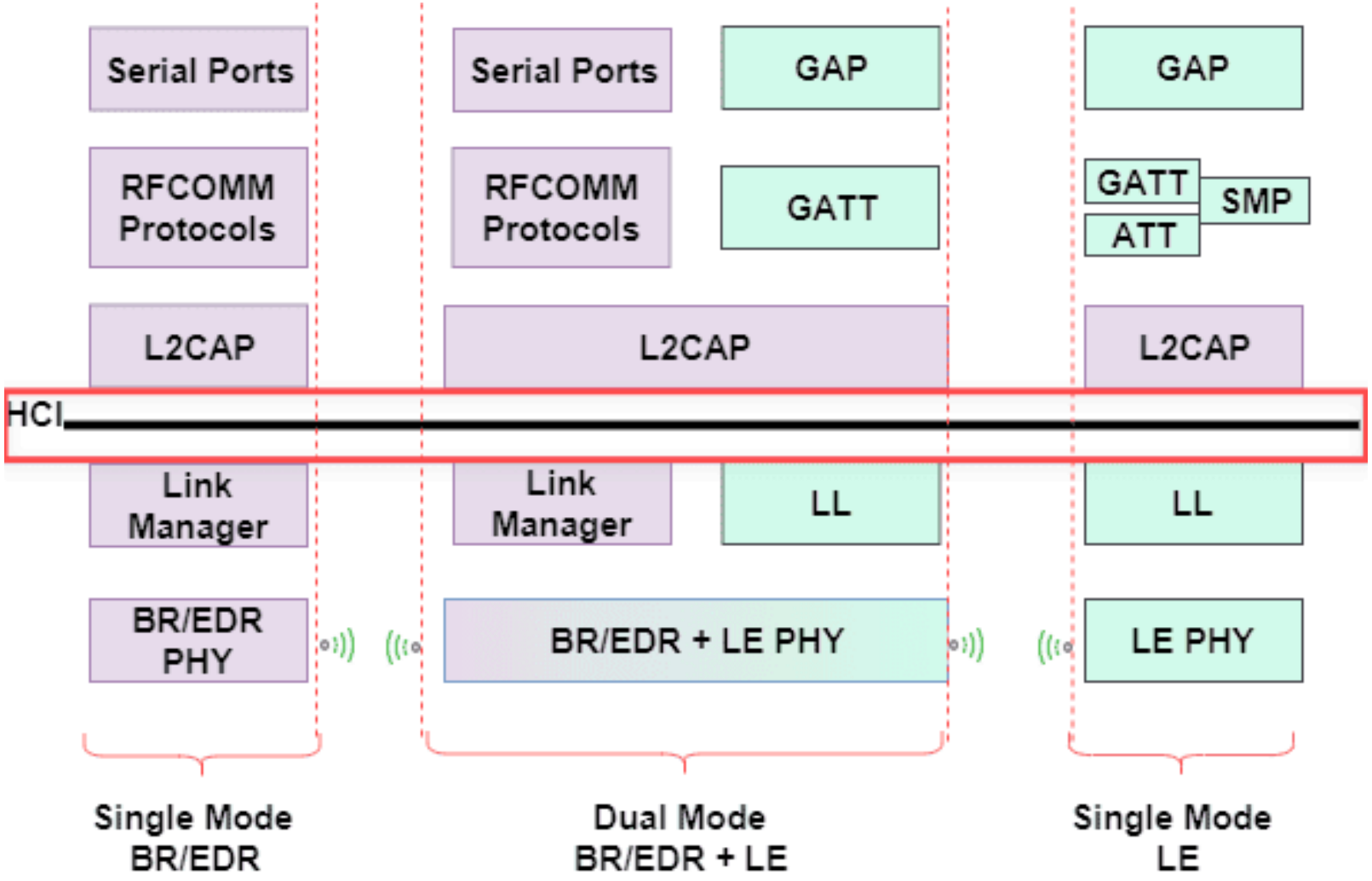
3. Multiplexing

L2CAP decides:

> Which packet belongs to which app?

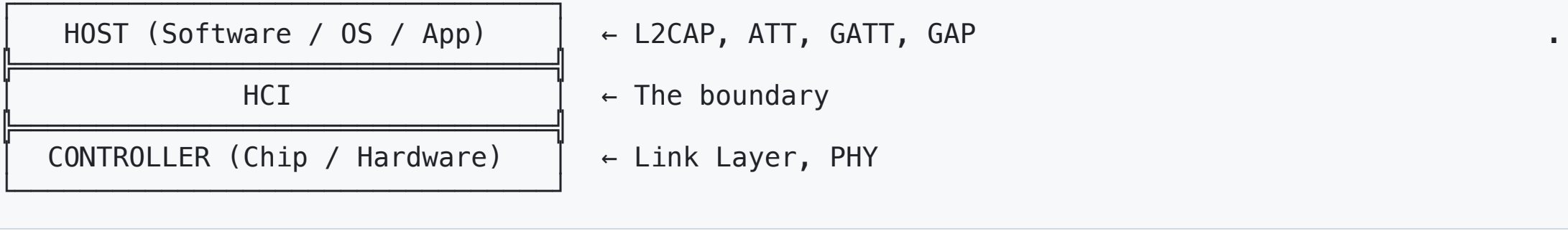
It labels and routes packets correctly.

Like sorting mail by department.


Serial Ports
RFCOMM Protocols
L2CAP
HCI
Link Manager
BR/EDR PHY
Serial Ports
RFCOMM Protocols
GAP
GATT
L2CAP
Link Manager
LL
BR/EDR + LE PHY
GAP
GATT
SMP
ATT
L2CAP
LL
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE

HCI

HCI is the **boundary layer** between software (Host) and hardware (Controller).



> Think of HCI as the **USB cable between your computer and a Bluetooth dongle**:
> the computer (Host) sends commands, the dongle (Controller) executes them on the radio.

1. This is why you need Bluetooth drivers on your computer (OS) —
   they implement the HCI interface to communicate with the Bluetooth hardware.
2. HCI commands include:
    - Scan for devices
    - Connect to a device
    - Send data
    - Manage power states

Why HCI Exists

Without HCI, Bluetooth software would be tightly coupled to specific hardware.

With HCI:

- The **Host** (OS/App) is hardware-independent
- The **Controller** (chip) can be swapped without changing the software
- Standard commands and events are defined by the Bluetooth spec

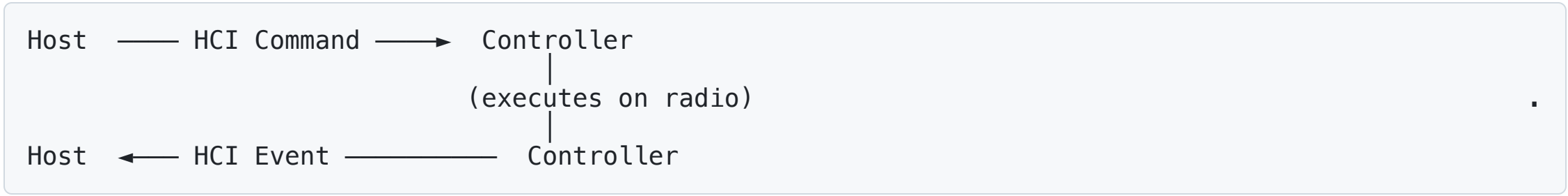
> Same Host stack works with any Bluetooth chip that speaks HCI.

Comparison with the Wifi Dongle

A typical USB Wi-Fi adapter contains:

- MAC processing (partial or full)
- Hardware encryption engine (WPA2 / WPA3)
- PHY (Radio layer)
- Embedded firmware
- Sometimes TCP/IP offloading features

> A USB Wi-Fi dongle typically performs **more networking logic in hardware** than a Bluetooth dongle, which mainly implements the controller side of the stack.



HCI Command examples:

- `HCI_LE_Set_Advertising_Parameters` → Start advertising
- `HCI_LE_Create_Connection` → Initiate connection
- `HCI_LE_Encrypt` → Encrypt data

HCI Event examples:

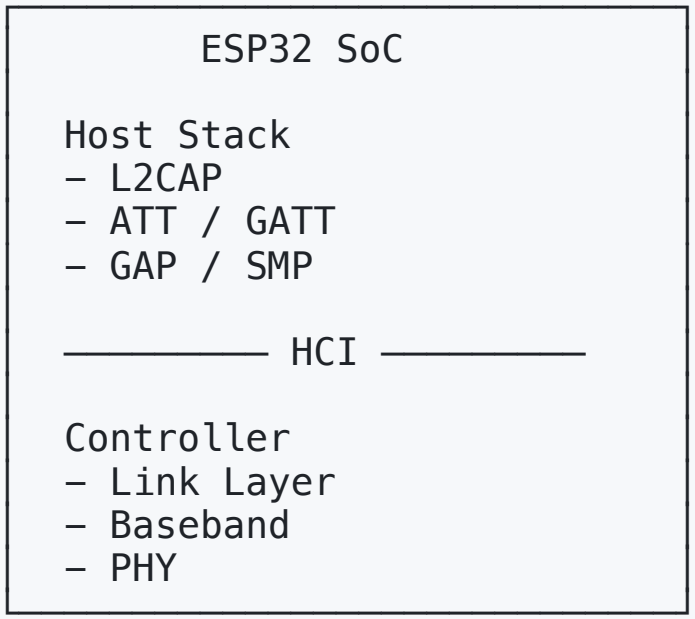
- `HCI_LE_Connection_Complete` → Connection established
- `HCI_LE_Advertising_Report` → Device found during scan

HCI Transport Options

Transport	Used In
UART	Embedded systems, modules
USB	PC Bluetooth dongles
SDIO	Wi-Fi+BT combo chips
SPI	Low-power MCU environments

> On smartphones, HCI runs internally between the OS and the Bluetooth chip — invisible to the developer.

ESP32 integrates both sides on one chip.



> HCI exists logically (software boundary), not as a physical bus.

ESP32 natively contains both Host + Controller stacks,
but you can configure it to expose only the Controller over HCI,
so an external system provides the Host stack — just like a USB dongle.

- This is useful for custom applications where you want to implement your own Host logic on a separate microcontroller or processor.

Link Layer (LL and LMP) — Same Goal, Different Design

that manages the radio link.

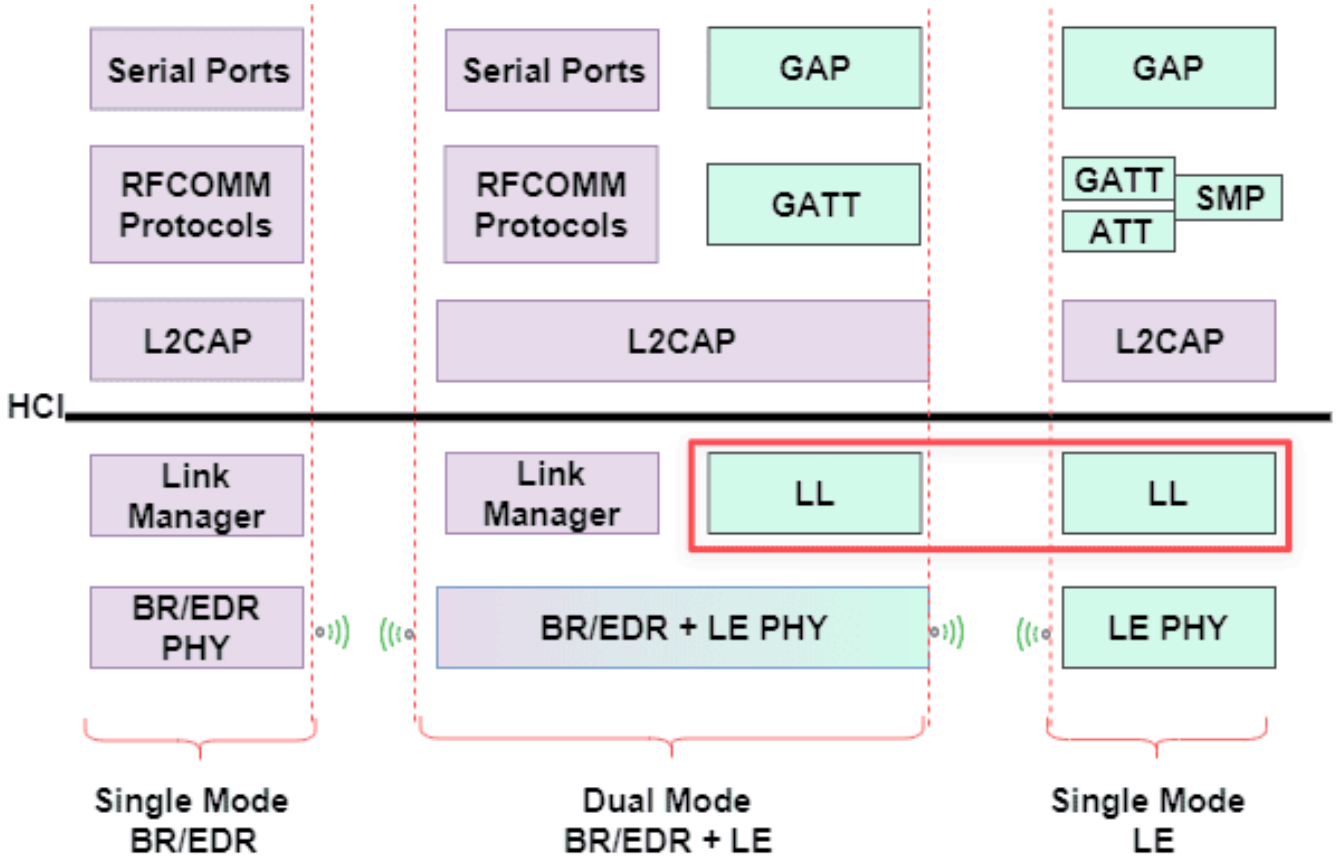
	BLE	Classic Bluetooth
Layer name	**Link Layer (LL)**	**Link Manager Protocol (LMP)**
Location	Controller (in chip)	Controller (in chip)
Sits above	PHY	Baseband / PHY
Core purpose	Manage the radio link	Manage the radio link

> They are **not** the same — but they share the same fundamental responsibility.

What LL and LMP Share (Common to Both)

Both LL and LMP handle:

- **Connection setup** — Establish the radio link between two devices
- **Connection maintenance** — Keep the link alive
- **Encryption** — Secure data at the link level
- **Disconnection** — Terminate the link cleanly
- **Channel scheduling** — Decide which channel to use next and when to hop (PHY executes the actual tuning)
- **Packet framing** — Define how bits are structured for transmission


Serial Ports
RFCOMM Protocols
L2CAP
HCI
Link Manager
BR/EDR PHY
Serial Ports
RFCOMM Protocols
GAP
GATT
L2CAP
Link Manager
LL
BR/EDR + LE PHY
GAP
GATT
SMP
ATT
L2CAP
LL
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE

Link Layer (LL) — BLE

LL is the BLE Controller layer, designed from scratch for **ultra-low energy** operation.

> LL is intentionally lean — it does only what is needed to manage a BLE radio link with minimal power.

LL and GAP — Who Does What?

```
GAP (Host layer)                    LL (Controller layer)
──────────────────────              ──────────────────────────────
"Scan for devices"          →       listen on ch 37/38/39
"Advertise as sensor"       →       broadcast PDU on fixed channels
"Connect to AA:BB:CC"       →       send CONNECT_IND,
                                     assign Access Address, start hopping
"Disconnect"                →       send LL_TERMINATE_IND
```

GAP decides the policy. LL executes it on the radio.

- GAP is the **brain** — who to connect, when, and under what security rules
- LL is the **hands** — actually managing the radio link

BLE-Specific LL Features

- **Advertising channels** — 3 fixed channels (37, 38, 39) for discovery
- **Data channels** — 37 hopping channels (0–36) after connection
- **Access Address** — 4-byte random session token per connection (not BD_ADDR)
- **Ultra-low power** — radio off between connection intervals
- **Adaptive hopping** — skips interference-heavy channels

BLE Packet Structure (LL)

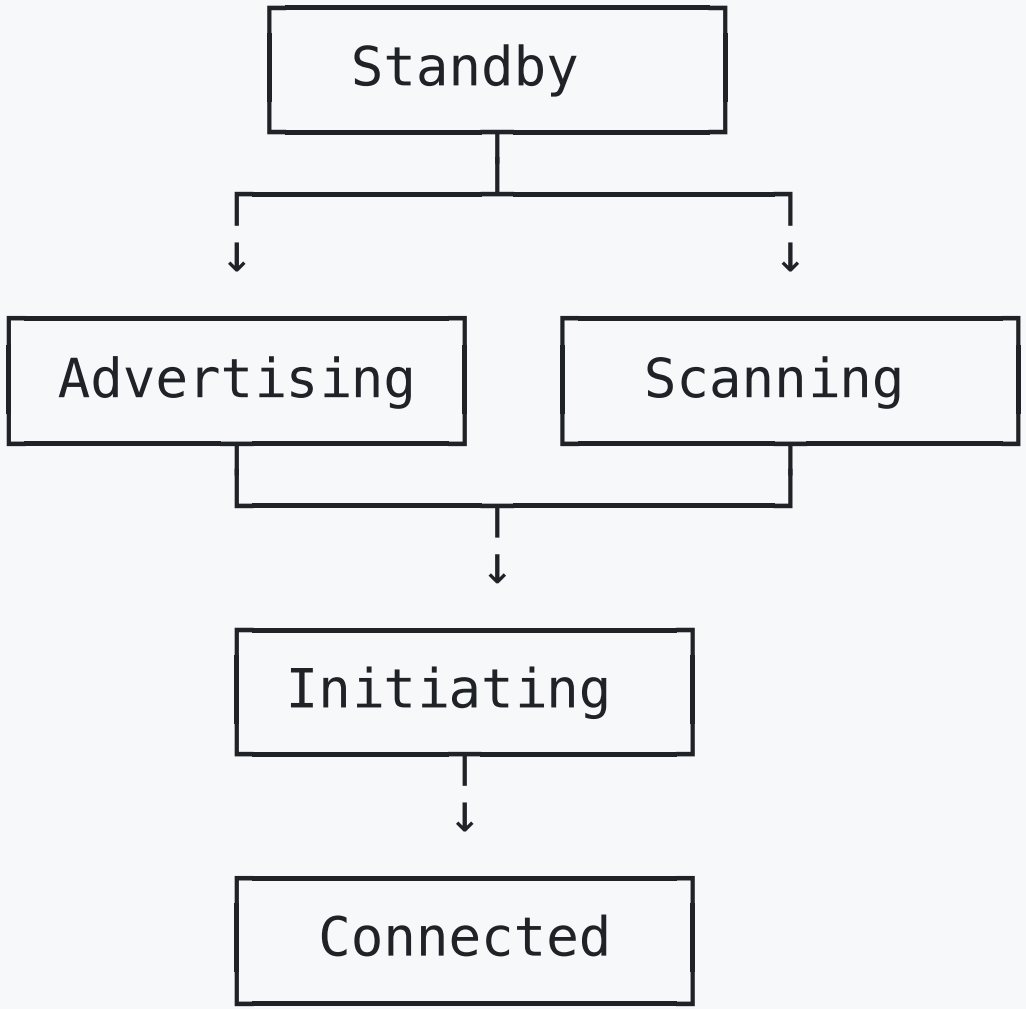
- **Preamble**: Sync pattern (10101010...)
- **Access Address**: 4-byte session token — **NOT BD_ADDR**
- **PDU**: Advertising payload or data channel payload
- **CRC**: 3-byte error detection

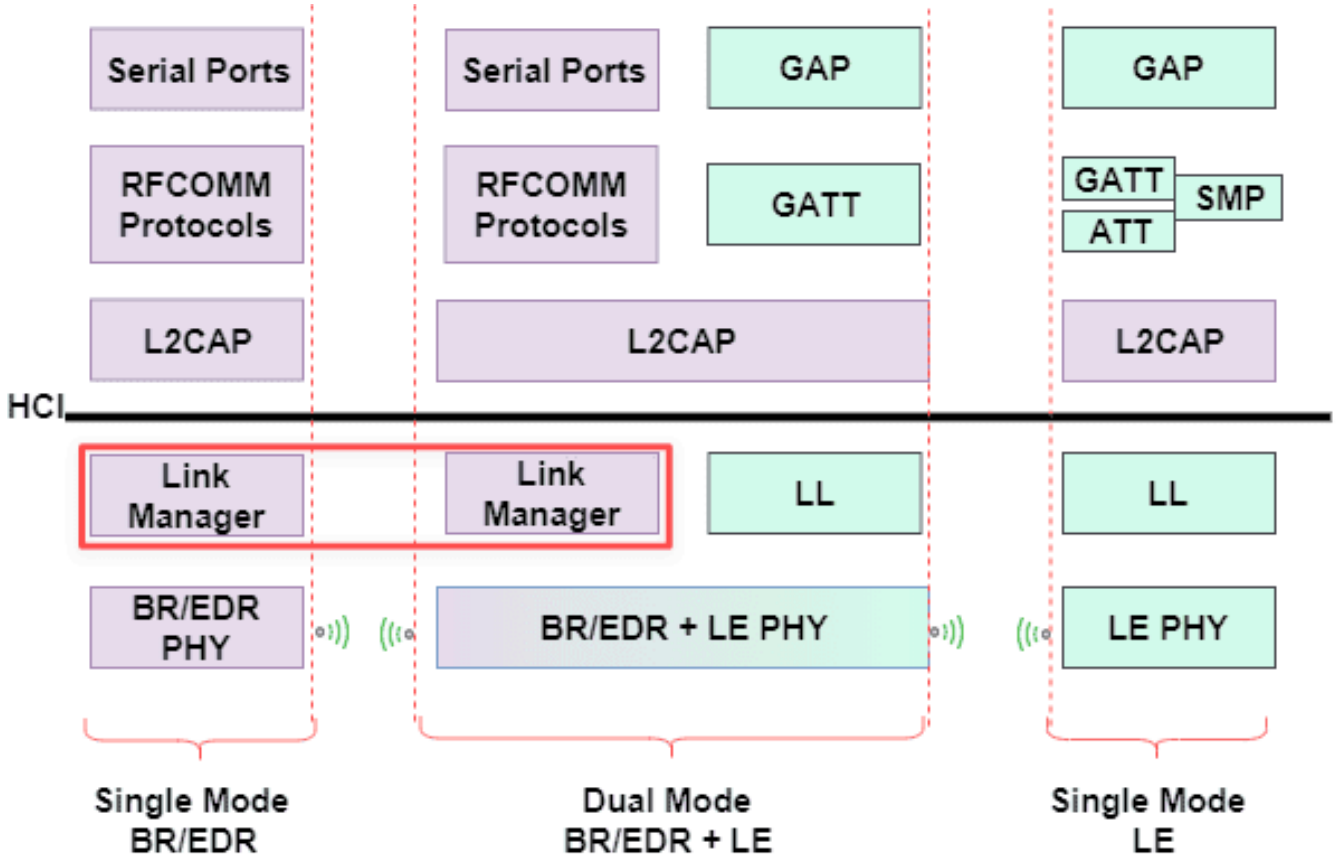
Access Address vs BD_ADDR

	Access Address	BD_ADDR
Size	4 bytes	6 bytes
Purpose	Identifies a BLE connection (session token)	Identifies a device
Scope	Valid only during one connection session	Globally unique (or randomized)
Who sets it	Central generates randomly at connection setup	Assigned by manufacturer / GAP
Where used	Every LL packet header	GAP advertising, pairing, scan

> Access Address distinguishes simultaneous BLE connections nearby —
> it has nothing to do with identifying who the device is.

LL States




Serial Ports
RFCOMM Protocols
L2CAP
HCI
Link Manager
BR/EDR PHY
Serial Ports
RFCOMM Protocols
GAP
GATT
L2CAP
Link Manager
LL
BR/EDR + LE PHY
GAP
GATT
SMP
ATT
L2CAP
LL
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE

Link Manager Protocol (LMP) — Classic Bluetooth

LMP is the Classic Bluetooth Controller layer, designed for **sustained, high-bandwidth** connections (audio streaming, file transfer).

> LMP is more complex than LL —
> it must handle audio channels, role switching, and richer power management.

LMP-Specific Features

- **Master/Slave role switching** — devices can swap roles dynamically
- **Power modes** — Sniff, Hold, Park (fine-grained power control)
- **QoS negotiation** — bandwidth and latency guarantees for audio
- **Authentication** — PIN / link key verification
- **SCO/eSCO links** — dedicated synchronous channels for voice/audio
- **79-channel FHSS** — fixed 1600 hops/sec

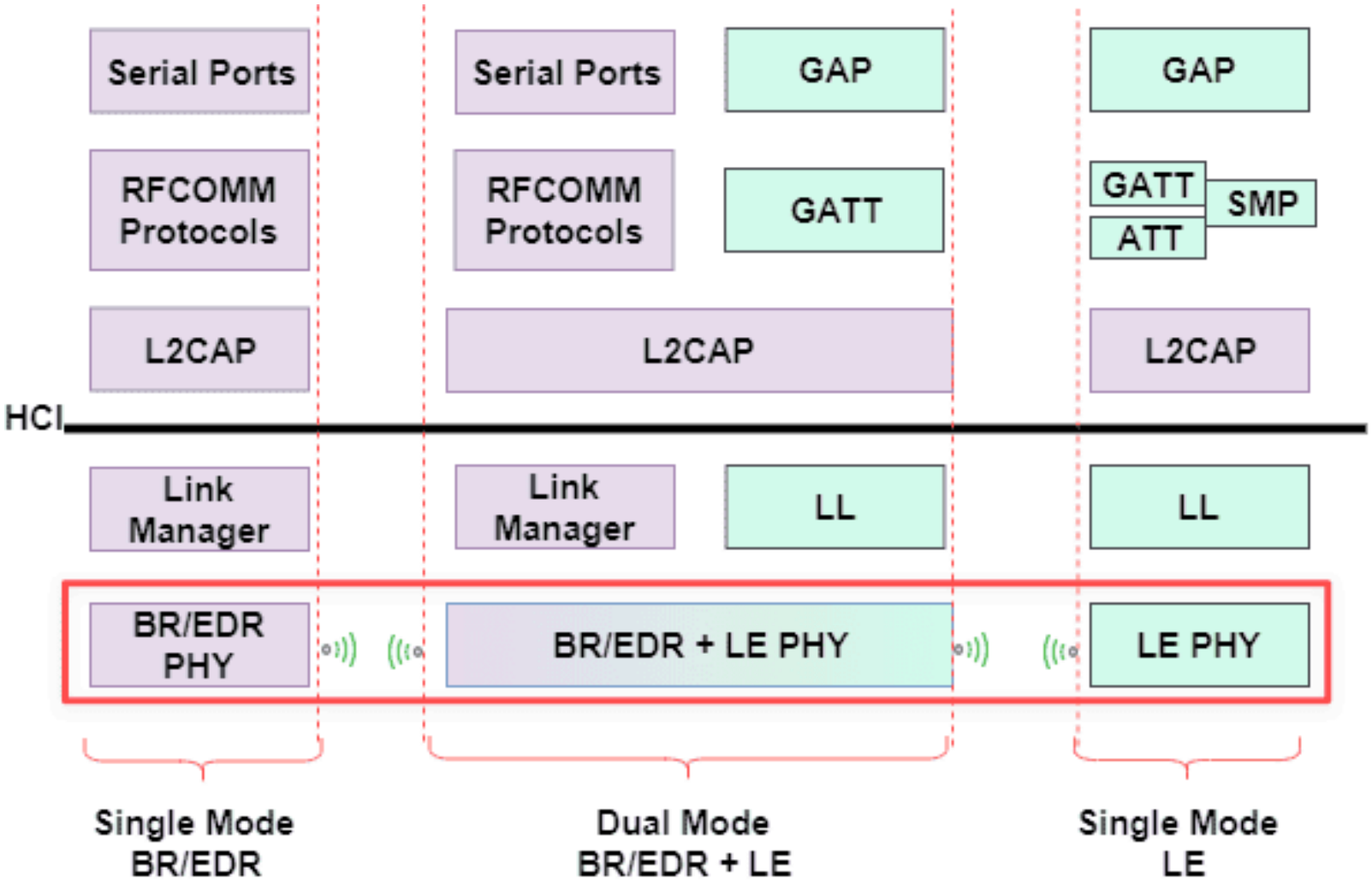
LMP Packet Structure

- **Opcode**: identifies the control command (e.g., `LMP_encryption_mode_req`)
- **Parameters**: link settings, keys, timing values
- **Payload**: optional control data

Exchanged peer-to-peer between two Link Managers to negotiate the link.

LL vs LMP — Key Differences

Aspect	LL (BLE)	LMP (Classic)
Design goal	Ultra-low power	High throughput / audio
Channels	40 ch (37 data + 3 adv)	79 ch (all data)
Hop rate	Adaptive (slow, per interval)	Fixed 1600 hops/sec
Power modes	Radio off between intervals	Sniff / Hold / Park
Role model	Central / Peripheral	Master / Slave
Audio support	LE Audio (BLE 5.2+)	SCO / eSCO (native)
Discovery	Advertising on fixed channels	Inquiry / Page procedure
Complexity	Simpler, stateless	Richer, stateful


Serial Ports
RFCOMM Protocols
L2CAP
HCI
Link Manager
BR/EDR PHY
Serial Ports
RFCOMM Protocols
GAP
GATT
L2CAP
Link Manager
LL
BR/EDR + LE PHY
GAP
GATT
SMP
ATT
L2CAP
LL
LE PHY
Single Mode BR/EDR
Dual Mode BR/EDR + LE
Single Mode LE

PHY Layer

The **Physical Layer (PHY)** is the lowest layer of the Bluetooth stack —
it handles actual radio **signal transmission and reception**.

> Think of PHY as the **radio hardware**:
> it converts 0s and 1s into radio waves, and radio waves back into 0s and 1s.

What PHY Does

- Transmits and receives **raw bits** over the 2.4 GHz radio
- **Tunes to the correct frequency channel** when instructed by LL
- Defines **modulation scheme** — how bits are encoded as radio signals
- Defines **transmission power** and **receiver sensitivity**
- Operates completely in **hardware** (inside the Bluetooth chip)

What is Frequency Hopping?

Instead of transmitting on a **single fixed frequency**, Bluetooth rapidly switches between many channels.

```
Time:  t1      t2      t3      t4      t5
        |       |       |       |       |
Freq:  ch5 → ch23 → ch11 → ch36 → ch2 → ...
```

Why? The 2.4 GHz band is crowded — Wi-Fi, microwaves, and other devices all share it.

> Hopping avoids interference by never staying on one channel long enough to be disrupted.

Frequency Hopping — LL and PHY Work Together

Frequency hopping is **not PHY alone** — it is a shared responsibility:

```
LL  →  scheduler: decides WHICH channel and WHEN to hop
       (channel map, hop sequence, timing)

PHY →  executor: physically tunes to that frequency
       (modulation, signal TX/RX on the selected channel)
```

> LL says: "next packet on channel 23"
> PHY does: switches to 2.446 GHz and transmits

Why Does BLE Use Fewer Channels?

BLE uses **40 channels** instead of Classic's 79 — but the total bandwidth is similar:

```
Classic:  79 channels × 1 MHz wide  =  79 MHz total
BLE:      40 channels × 2 MHz wide  =  80 MHz total
```

BLE trades more channels for **wider channels** — by design:

- Wider channel → better noise tolerance, packet sent faster
- Faster packet → radio turns off sooner → **less power**
- 3 fixed advertising channels (37/38/39) are placed in the **gaps between Wi-Fi channels 1/6/11** to avoid interference

> BLE's goal is not maximum throughput — it's **send fast, then sleep**.

Channel Map: Classic vs BLE

Aspect	Classic Bluetooth	BLE
Channels	79 channels (1 MHz wide)	40 channels (2 MHz wide)
Hop rate	1600 hops/sec (fixed)	Adaptive (per connection interval)
Data channels	79	37 (ch 0–36)
Adv channels	None	3 fixed (ch 37, 38, 39)
Modulation	GFSK / π/4-DQPSK / 8DPSK	GFSK only

BLE Channel Map

```
BLE 40 Channels (2 MHz each)

 Ch: 37  0  1  2 ...11 38 12 ...36 39
     |   |  |  |      |  |      |  |
     AD  D  D  D  ... AD  D ... D  AD

 AD = Advertising channel (fixed: 37, 38, 39)
 D  = Data channel (0–36, used after connection)
```

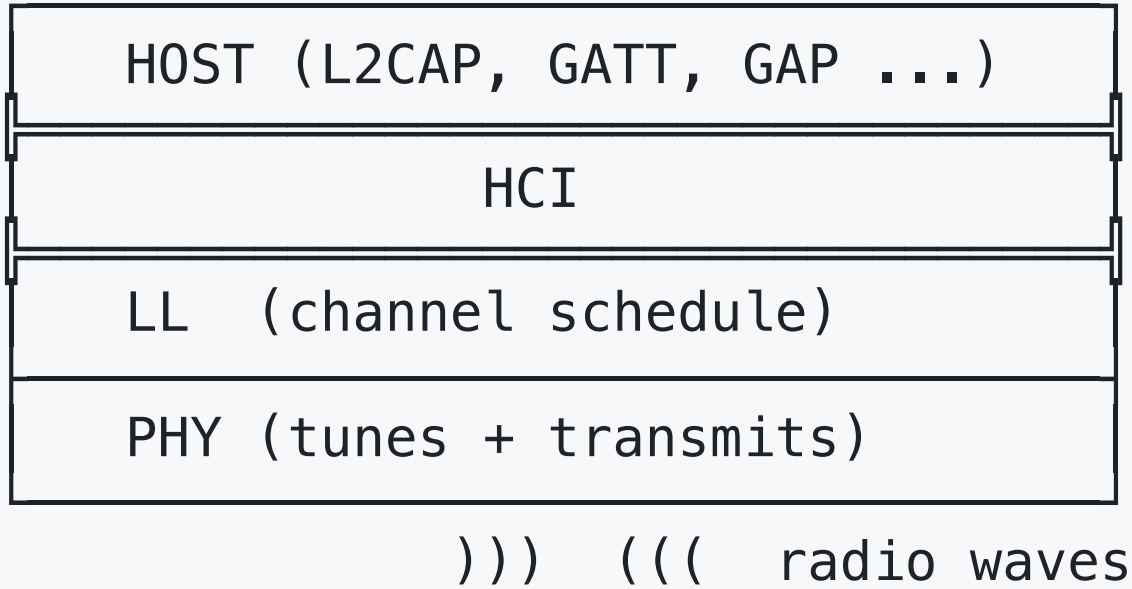
- Advertising channels are **fixed** — devices always know where to find each other
- Data channels **hop** after connection — LL schedules, PHY executes

BLE PHY Modes (BLE 5.0+)

PHY Mode	Speed	Range	Use Case
1M PHY	1 Mbps	~50m	Standard BLE (default)
2M PHY	2 Mbps	~30m	Fast data, short range
Coded PHY	125–500 kbps	~200–400m	Long range (BLE 5.0+)

> PHY mode is negotiated after connection — both devices must support the same mode.

PHY in Context



Frequency hopping = **LL schedules + PHY executes**. They work together.